

CO2-AT3-Debugging And Optimization

CSA-0603-DESIGN OF ANALYSIS AND ALGORITHMS

CHADALAVADA VENKATARAMAYOGI

Register Number: 192424319

Q7. Debugging Insertion Sort Code

The following Insertion Sort code fails to correctly place elements in sorted order. Identify errors in shifting logic and loop conditions. Explain why incorrect insertion occurs. Debug the algorithm step-by-step to ensure correct functionality. Optimize the code for better readability. Analyze the time complexity. Rewrite the corrected version with proper structure and comments.

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j > 0 and arr[j] > key: # ERROR  
            arr[j+1] = arr[j]  
            j -= 1  
        arr[j] = key # ERROR  
    return arr
```

Aim

To identify and correct the errors in the given Insertion Sort program, debug the algorithm, optimize it for readability, and analyze its time complexity.

Algorithm

1. Read the input array.
2. Start from the second element of the array.
3. Store the current element as key.
4. Compare the key with the previous elements.
5. Shift all elements greater than the key one position to the right.
6. Insert the key at its correct position.
7. Repeat until all elements are sorted.

8. Display the sorted array.

Given Incorrect Code

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j > 0 and arr[j] > key: # ERROR  
            arr[j+1] = arr[j]  
            j -= 1  
        arr[j] = key # ERROR  
    return arr
```

Errors Identified

Error 1: Incorrect Loop Condition

```
while j > 0 and arr[j] > key
```

Problem:

- The loop stops when j becomes 0.
- Therefore, the first element (arr[0]) is never compared.

Correct Statement

```
while j >= 0 and arr[j] > key
```

Error 2: Incorrect Insertion Position

```
arr[j] = key
```

Problem:

- After shifting elements, j is decreased by one.
- The correct position for insertion is j + 1.

Correct Statement

```
arr[j + 1] = key
```

Corrected Program

```
def insertion_sort(arr):  
    # Traverse through the array
```

```
for i in range(1, len(arr)):
    key = arr[i]      # Current element
    j = i - 1
    # Shift elements greater than key
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1
    # Insert key into the correct position
    arr[j + 1] = key
return arr

# Input
arr = list(map(int, input("Enter elements: ").split()))

# Function call
sorted_array = insertion_sort(arr)

# Output
print("Sorted Array:", sorted_array)
```

Sample Input

Enter elements: 8 4 6 2 7

Sample Output

Sorted Array: [2, 4, 6, 7, 8]

Step-by-Step Debugging

Initial Array

[8, 4, 6, 2, 7]

Pass 1 (Key = 4)

- Compare $8 > 4 \rightarrow$ Shift 8
- Insert 4

[4, 8, 6, 2, 7]

Pass 2 (Key = 6)

- Compare $8 > 6 \rightarrow$ Shift 8
- Insert 6

[4, 6, 8, 2, 7]

Pass 3 (Key = 2)

- Shift 8
- Shift 6
- Shift 4
- Insert 2

[2, 4, 6, 8, 7]

Pass 4 (Key = 7)

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key  
    return arr  
arr = list(map(int, input("Enter elements: ").split()))  
sorted_arr = insertion_sort(arr)  
print("Sorted Array:", sorted_arr)
```

```
... Enter elements: 8 4 9 7  
Sorted Array: [4, 7, 8, 9]
```

- Shift 8
- Insert 7

[2, 4, 6, 7, 8]

Optimizations

- ❖ Corrected the loop condition from $j > 0$ to $j \geq 0$ so that the first element is also compared.
- ❖ Changed the insertion statement from `arr[j] = key` to `arr[j + 1] = key` to place the key in the correct position.
- ❖ Added meaningful comments to improve code readability and understanding.
- ❖ Used proper indentation and formatting to follow Python coding standards.
- ❖ Chose descriptive variable names (key and j) for better code clarity.
- ❖ Removed logical errors to ensure the algorithm sorts the array correctly.
- ❖ Maintained an in-place sorting approach, requiring only **O(1)** extra space.
- ❖ Preserved the best-case time complexity of **O(n)** for already sorted arrays and **O(n²)** for average and worst cases.
- ❖ Improved the overall maintainability and readability of the program without changing the core Insertion Sort algorithm.

Time Complexity Analysis

Case	Time Complexity
Best Case (Already Sorted)	$O(n)$
Average Case	$O(n^2)$
Worst Case (Reverse Sorted)	$O(n^2)$

Space Complexity

O(1)

(In-place sorting algorithm.)

Result

The errors in the Insertion Sort program were successfully identified and corrected. The algorithm now correctly sorts the array by comparing all necessary elements and inserting each key into its proper position. The optimized version is more readable, maintainable, and runs with $O(n)$ time in the best case and $O(n^2)$ time in the average and worst cases, using $O(1)$ extra space.

